Database Lab : Degraded Query Plan Triage & Optimizer Statistics Refresh

Duration : 60 Min.

Scenario

As a Database Performance Architect at NIIT, an overnight batch query regressed from 30 seconds to 45 minutes due to stale optimizer statistics and missing composite indexes. You must create the schema in PostgreSQL, run `ANALYZE` to refresh statistics, create the composite index, and record optimization metrics.

Input Details

The environment has been pre-configured with the following local database platform instances:

- **Database Host:** `localhost` (Ubuntu Linux VM)
- **PostgreSQL Service:** `localhost:5432` | DB: `postgres` | User: `postgres` | Password: `PostgresPassword123!` | CLI: `psql`

Username & Naming Conventions

Your candidate prefix is: **`<prefix>`** (e.g. `student-exam123`).

Resource naming: - **Orders Table:** `<prefix>_orders_perf` - **Composite Index:** `idx_<prefix>_orders_status_region` - **Performance Audit Table:** `<prefix>_query_opt_audit`

Task Objectives

Perform the following optimization steps directly via `psql`:

1. Create & Seed Orders Table

- Create table `<prefix>_orders_perf` with columns:
- `order_id SERIAL PRIMARY KEY`
- `status VARCHAR(20) NOT NULL`
- `region VARCHAR(20) NOT NULL`
- `amount NUMERIC(10,2) NOT NULL`
- Insert at least 500 records with diverse status (`PENDING`, `COMPLETED`) and region values.

2. Refresh Optimizer Statistics

- Execute `ANALYZE <prefix>_orders_perf;` to update `pg_statistic`.

3. Create Composite Index

- Create composite index `idx_<prefix>_orders_status_region` on `<prefix>_orders_perf (status, region)`.

4. Create Performance Audit Table

- Create table `<prefix>_query_opt_audit (id SERIAL PRIMARY KEY, target_table VARCHAR(50), optimization VARCHAR(50), plan_operator VARCHAR(30), verified_by VARCHAR(50))`.

- Insert a record: `INSERT INTO <prefix>_query_opt_audit VALUES (1, '<prefix>_orders_perf', 'ANALYZE & INDEX', 'Index Scan', '<prefix>');`.

Verification

Once completed, run the verification script:

```
python run.py
```

The driver connects to PostgreSQL, checks `pg_indexes`, verifies `pg_stat_user_tables.last_analyze`, and inspects `<prefix>_query_opt_audit`.

Grading Criteria

Your performance will be evaluated based on the following test cases:

Test Case	Requirement	Marks
TC1	Table (_orders_perf) Creation & Data Seeding	3 Marks
TC2	Optimizer Statistics Refresh (ANALYZE) Execution	3 Marks
TC3	Composite Index (idx__orders_status_region) Creation	3 Marks
TC4	Performance Audit Table (_query_opt_audit) Creation	3 Marks
TC5	Target Plan Operator (Index Scan) Transformation	2 Marks
TC6	Query Predicate (status, region) Coverage Verification	2 Marks
TC7	Catalog Statistics Verification in pg_stat_user_tables	2 Marks
TC8	End-to-End Query Optimization Verification	2 Marks

Total Score: 20 Marks

Important Notes

- Replace `<prefix>` with your candidate prefix.
- All actions must be performed on the live PostgreSQL instance.